



Lux ID: Decentralized Identity and Universal Access Management

A Five-Year Journey from

Web2 to Web3 Identity

Zach Kelling

Lux Industries

2020

Abstract

We present Lux ID, a comprehensive decentralized identity (DID) and Identity Access Management (IAM) system that bridges Web2 and Web3 authentication paradigms. Initially deployed in October 2020, Lux ID has evolved over five years to support multi-protocol authentication (OAuth 2.0, OpenID Connect, SAML, CAS, LDAP), advanced security features (WebAuthn, TOTP, MFA), and blockchain-native identity via the `did:lux:address` format. The October 2025 revision introduces post-quantum cryptographic credentials, hardware security key enhancements, and seamless integration with the Lux blockchain ecosystem. This paper provides the complete DID specification, architectural details, security analysis, performance benchmarks, and comparisons with existing identity systems. Lux ID demonstrates that production-grade decentralized identity can coexist with traditional IAM requirements while maintaining superior security, privacy, and user experience.

Keywords: Decentralized Identity, DID, IAM, OAuth 2.0, WebAuthn, Post-Quantum Cryptography, Blockchain Identity, Multi-Protocol Authentication

Version History

- **v2020.10** (October 2020): Initial implementation based on Casdoor, core IAM features, `did:lux:address` format introduced
- **v2021.06** (June 2021): Added WebAuthn support for passwordless authentication
- **v2022.03** (March 2022): LDAP and RADIUS integration for enterprise deployments
- **v2022.11** (November 2022): Multi-factor authentication (MFA) and TOTP implementation
- **v2023.07** (July 2023): MetaMask and Web3 wallet integration for blockchain authentication
- **v2024.02** (February 2024): SCIM protocol support for automated provisioning
- **v2025.10** (October 2025): **Current revision** — Post-quantum credentials (CRYSTALS-Dilithium), enhanced hardware security key support, full Lux ecosystem integration

Contents

1	Introduction	5
1.1	The Identity Crisis in Web3	5
1.2	Design Philosophy	5
1.3	Contributions	5
2	Lux DID Specification	6
2.1	DID Format	6
2.2	Network Identifiers	6
2.3	Address Formats	6
2.4	DID Document Structure	6
2.5	DID Operations	7
2.5.1	Create	7
2.5.2	Read (Resolve)	8
2.5.3	Update	8
2.5.4	Deactivate	8

3	IAM Architecture	8
3.1	System Overview	8
3.2	Core Components	8
3.2.1	Backend (Go/Beego)	8
3.2.2	Frontend (React + TypeScript)	8
3.2.3	Data Layer	9
3.3	Multi-Protocol Support	9
3.3.1	OAuth 2.0 & OpenID Connect	9
3.3.2	SAML 2.0	9
3.3.3	LDAP & RADIUS	9
3.3.4	WebAuthn & FIDO2	9
3.3.5	Web3 Wallet Authentication	10
4	Security Features	10
4.1	Cryptographic Primitives	10
4.1.1	Traditional Cryptography	10
4.1.2	Post-Quantum Cryptography (v2025.10)	10
4.2	Multi-Factor Authentication	11
4.2.1	TOTP (Time-Based One-Time Passwords)	11
4.2.2	SMS & Email Verification	11
4.2.3	Hardware Security Keys	11
4.3	Threat Model	11
4.3.1	Assumptions	11
4.3.2	Security Guarantees	11
4.3.3	Attack Surface Reduction	12
5	Integration with Lux Ecosystem	12
5.1	Lux Node	12
5.2	Lux Wallet	12
5.3	Lux Bridge	12
5.4	Lux Exchange	12
6	Cross-Chain Identity Verification	13
6.1	Multi-Chain Proof of Ownership	13
6.2	Decentralized Identity Aggregation	13
7	Privacy Considerations	13
7.1	Data Minimization	13
7.2	Selective Disclosure	13
7.3	Zero-Knowledge Proofs (Planned)	14
7.4	Right to be Forgotten	14
8	Performance and Scalability	14
8.1	Production Metrics (5 Years)	14
8.2	Scalability Architecture	14
8.2.1	Horizontal Scaling	14
8.2.2	Caching Strategy	14
8.2.3	Rate Limiting	15
8.3	Benchmarking	15
9	Comparison with Other DID Systems	15
9.1	Key Differentiators	15

10 Future Work	16
10.1 Short-Term (2025-2025)	16
10.2 Medium-Term (2025-2027)	16
10.3 Long-Term (2027+)	16
11 Conclusion	16
A DID Method Specification	17
A.1 Method Name	17
A.2 Method-Specific Identifier	17
A.3 DID Syntax	18
A.4 CRUD Operations	18
A.4.1 Create	18
A.4.2 Read (Resolve)	18
A.4.3 Update	18
A.4.4 Deactivate	19
B API Reference	19
B.1 Authentication Endpoints	19
B.2 Management Endpoints	19
C Deployment Guide	20
C.1 Docker Compose (Development)	20
C.2 Kubernetes (Production)	20

1 Introduction

1.1 The Identity Crisis in Web3

The transition from Web2 to Web3 has exposed fundamental tensions in identity management:

- **Fragmentation:** Users manage dozens of private keys across chains and applications
- **Recovery:** Lost keys mean permanent loss of identity and assets
- **Interoperability:** No universal standard for cross-chain identity verification
- **Privacy:** On-chain activities are pseudonymous but not anonymous
- **Compliance:** Regulatory requirements (KYC/AML) conflict with decentralization ideals

Traditional Web2 identity systems (OAuth, SAML, LDAP) offer robust security and user experience but rely on centralized authorities. Blockchain-native approaches (ENS, Unstoppable Domains) provide decentralization but lack enterprise features and multi-protocol support.

1.2 Design Philosophy

Lux ID bridges this gap through a **pragmatic hybrid approach**:

1. **Universal Compatibility:** Support all major authentication protocols (OAuth 2.0, OIDC, SAML, CAS, LDAP, RADIUS, WebAuthn)
2. **Blockchain Native:** First-class support for wallet-based authentication and DID resolution
3. **Progressive Decentralization:** Users can start with traditional accounts and gradually migrate to self-sovereign identity
4. **Enterprise Ready:** Multi-tenancy, RBAC, audit logging, and compliance features
5. **Quantum Resistant:** Post-quantum cryptographic primitives for future-proof security

1.3 Contributions

This paper makes the following contributions:

- **DID Specification:** Complete specification of the `did:lux:address` format
- **Multi-Protocol Architecture:** Design patterns for bridging Web2 and Web3 identity
- **Security Analysis:** Threat model and cryptographic guarantees
- **Performance Benchmarks:** Production metrics from 5 years of deployment
- **Integration Patterns:** Blueprints for ecosystem-wide identity verification
- **Post-Quantum Migration:** Strategy for transitioning to quantum-resistant credentials

2 Lux DID Specification

2.1 DID Format

The Lux DID method follows W3C Decentralized Identifiers (DIDs) v1.0 specification [1]. The general format is:

`did:lux:<network>:<address>`

Where:

- `did` — DID scheme identifier (constant)
- `lux` — DID method name
- `<network>` — Network identifier (optional, defaults to mainnet)
- `<address>` — Lux blockchain address or public key hash

2.2 Network Identifiers

Network	Identifier	Example
Mainnet	(omitted)	<code>did:lux:X-lux1abc...</code>
Testnet	<code>testnet</code>	<code>did:lux:testnet:X-lux1abc...</code>
Local	<code>local</code>	<code>did:lux:local:X-lux1abc...</code>
Custom Appchain	<code><appchain-id></code>	<code>did:lux:2oYMBNV4eNH...</code>

Table 1: Lux Network Identifiers

2.3 Address Formats

Lux supports multiple address formats across chains:

- **X-Chain (UTXO):** `X-lux1<bech32>` (AVM address)
- **C-Chain (EVM):** `0x<hex>` (Ethereum-compatible)
- **P-Chain (Platform):** `P-lux1<bech32>` (validator/staking)

All formats can be used in DID identifiers:

```
1 did:lux:X-lux1qzr2v3dhq0hgqkgdwq36z0z7eqkh5x2g5m
2 did:lux:0x742d35Cc6634C0532925a3b844Bc9e7595f0bEb
3 did:lux:P-lux1g65uqn6t77p656w64023nh8nd9updzmhx8ttv
```

Listing 1: Valid Lux DIDs

2.4 DID Document Structure

Each DID resolves to a DID Document conforming to W3C standards:

```
1 {
2   "@context": [
3     "https://www.w3.org/ns/did/v1",
4     "https://w3id.org/security/suites/ed25519-2020/v1",
5     "https://w3id.org/security/suites/x25519-2020/v1"
6   ],
```

```

7  "id": "did:lux:X-lux1qzr2v3dhq0hgqkgdwq36z0z7eqkh5x2g5m",
8  "controller": "did:lux:X-lux1qzr2v3dhq0hgqkgdwq36z0z7eqkh5x2g5m",
9  "verificationMethod": [
10   {
11     "id": "did:lux:X-lux1qzr2v3dhq0hgqkgdwq36z0z7eqkh5x2g5m#key-1",
12     "type": "Ed25519VerificationKey2020",
13     "controller": "did:lux:X-lux1qzr2v3dhq0hgqkgdwq36z0z7eqkh5x2g5m",
14     "publicKeyMultibase": "zH3C2AVvLMv6gmMnam3uVAjZpfkcJCwDwnZn6z3wXmqPV"
15   },
16   {
17     "id": "did:lux:X-lux1qzr2v3dhq0hgqkgdwq36z0z7eqkh5x2g5m#key-2",
18     "type": "Dilithium3VerificationKey2025",
19     "controller": "did:lux:X-lux1qzr2v3dhq0hgqkgdwq36z0z7eqkh5x2g5m",
20     "publicKeyMultibase": "zMxY...PQ-KEY" // Post-quantum key
21   }
22 ],
23 "authentication": [
24   "did:lux:X-lux1qzr2v3dhq0hgqkgdwq36z0z7eqkh5x2g5m#key-1",
25   "did:lux:X-lux1qzr2v3dhq0hgqkgdwq36z0z7eqkh5x2g5m#key-2"
26 ],
27 "assertionMethod": [
28   "did:lux:X-lux1qzr2v3dhq0hgqkgdwq36z0z7eqkh5x2g5m#key-1"
29 ],
30 "keyAgreement": [
31   {
32     "id": "did:lux:X-lux1qzr2v3dhq0hgqkgdwq36z0z7eqkh5x2g5m#key-3",
33     "type": "X25519KeyAgreementKey2020",
34     "controller": "did:lux:X-lux1qzr2v3dhq0hgqkgdwq36z0z7eqkh5x2g5m",
35     "publicKeyMultibase": "z6LSHs9GGnqk85isEBzzshkuVWrVKsRp24GnDuHk8QWkARMW"
36   }
37 ],
38 "service": [
39   {
40     "id": "did:lux:X-lux1qzr2v3dhq0hgqkgdwq36z0z7eqkh5x2g5m#id-hub",
41     "type": "IdentityHub",
42     "serviceEndpoint": "https://id.lux.network/hub/X-lux1qzr..."
43   },
44   {
45     "id": "did:lux:X-lux1qzr2v3dhq0hgqkgdwq36z0z7eqkh5x2g5m#oidc",
46     "type": "OpenIDConnectProvider",
47     "serviceEndpoint": "https://id.lux.network/oidc"
48   }
49 ]
50 }

```

Listing 2: Example DID Document

2.5 DID Operations

2.5.1 Create

Creating a DID involves:

1. Generate key pair (Ed25519, secp256k1, or Dilithium3)
2. Derive Lux address from public key
3. Construct DID: `did:lux:<address>`
4. Register DID Document on-chain (optional) or maintain off-chain registry

2.5.2 Read (Resolve)

DID resolution follows this priority:

1. **On-chain registry:** Check C-Chain smart contract for DID Document
2. **Lux ID service:** Query centralized resolver for federated identities
3. **Universal Resolver:** Fall back to DIF Universal Resolver [2]
4. **Derived from address:** Generate minimal DID Document from public key

2.5.3 Update

DID Document updates require proof of control:

- Sign update transaction with private key corresponding to DID
- Submit to on-chain registry or Lux ID service
- Version incremented with each update

2.5.4 Deactivate

Deactivation sets DID Document status to **deactivated**, preventing future authentication. Keys cannot be recovered after deactivation.

3 IAM Architecture

3.1 System Overview

Lux ID is built on a modular architecture supporting multiple authentication protocols:

3.2 Core Components

3.2.1 Backend (Go/Beego)

- **Controllers:** Handle HTTP requests for each protocol
- **Objects:** Business logic for users, applications, tokens
- **Authenticators:** Protocol-specific authentication (OAuth, SAML, WebAuthn)
- **IDP Integrations:** Social logins (Google, GitHub, MetaMask)
- **RBAC Engine:** Casbin-based permission enforcement

3.2.2 Frontend (React + TypeScript)

- Modern single-page application (SPA)
- Black theme optimized for Lux branding (#FFD700 gold accents)
- Responsive design for mobile and desktop
- Real-time session management

3.2.3 Data Layer

- **SQL Database:** User profiles, organizations, applications, permissions
- **Redis:** Session storage, rate limiting counters, temporary tokens
- **Blockchain:** DID registry, credential attestations (optional)

3.3 Multi-Protocol Support

3.3.1 OAuth 2.0 & OpenID Connect

Full implementation of OAuth 2.0 [3] and OIDC [4] flows:

- Authorization Code Flow (with PKCE)
- Implicit Flow
- Client Credentials Flow
- Resource Owner Password Credentials Flow
- Device Authorization Flow

Supported Endpoints:

```
1 GET /authorize # Authorization endpoint
2 POST /token # Token endpoint
3 GET /userinfo # UserInfo endpoint
4 GET /.well-known/openid-configuration # Discovery
5 GET /.well-known/jwks.json # JSON Web Key Set
```

3.3.2 SAML 2.0

Supports both Identity Provider (IdP) and Service Provider (SP) roles:

- Single Sign-On (SSO)
- Single Logout (SLO)
- Attribute statements (custom claims)
- POST and Redirect bindings

3.3.3 LDAP & RADIUS

Enterprise integration via:

- LDAP server for directory queries
- LDAP sync for importing users from Active Directory
- RADIUS server for VPN/Wi-Fi authentication

3.3.4 WebAuthn & FIDO2

Hardware security key support via WebAuthn [5]:

- Registration flow: Challenge → Credential Creation → Storage
- Authentication flow: Challenge → Assertion → Verification
- Resident keys for passwordless login
- User verification (biometric or PIN)

3.3.5 Web3 Wallet Authentication

Native blockchain authentication:

- **MetaMask**: Ethereum-compatible wallet via EIP-1193
- **WalletConnect**: Mobile wallet protocol
- **Web3Onboard**: Multi-wallet aggregator
- **Challenge-Response**: Sign nonce to prove ownership

Authentication Flow:

Algorithm 1 Web3 Wallet Authentication

- 1: **Client**: Request login with wallet address
 - 2: **Server**: Generate nonce, store in session
 - 3: **Server**: Return challenge message with nonce
 - 4: **Client**: Sign challenge with wallet's private key
 - 5: **Client**: Submit signature to server
 - 6: **Server**: Recover signer address from signature
 - 7: **Server**: Verify address matches claimed identity
 - 8: **Server**: Issue JWT access token
-

4 Security Features

4.1 Cryptographic Primitives

4.1.1 Traditional Cryptography

- **Password Hashing**: Argon2id (winner of Password Hashing Competition)
- **Session Tokens**: HMAC-SHA256 with 256-bit keys
- **JWT Signing**: RS256 (RSA-2048) or ES256 (ECDSA-P256)
- **Encryption**: AES-256-GCM for sensitive data at rest

4.1.2 Post-Quantum Cryptography (v2025.10)

To address quantum computing threats, Lux ID v2025.10 introduces:

- **CRYSTALS-Dilithium** [6]: Digital signatures (NIST PQC standard)
 - Public key: 1,952 bytes
 - Signature: 3,293 bytes
 - Security: NIST Level 3 (equivalent to AES-192)
- **CRYSTALS-Kyber** [7]: Key encapsulation mechanism
 - Public key: 1,568 bytes
 - Ciphertext: 1,568 bytes
 - Security: NIST Level 3

Hybrid Approach: DID Documents contain both classical (Ed25519) and post-quantum (Dilithium3) verification methods. Clients verify both signatures during a transition period.

4.2 Multi-Factor Authentication

4.2.1 TOTP (Time-Based One-Time Passwords)

RFC 6238 [8] compliant implementation:

- 6-digit codes, 30-second time window
- Compatible with Google Authenticator, Authy, 1Password
- QR code enrollment

4.2.2 SMS & Email Verification

- Pluggable providers: Twilio, SendGrid, AWS SES, custom SMTP
- Rate limiting: Max 5 codes per hour per user
- Code expiry: 10 minutes

4.2.3 Hardware Security Keys

Full WebAuthn/FIDO2 support:

- YubiKey, Google Titan, Feitian
- NFC, USB-A, USB-C, Lightning
- Biometric authentication (fingerprint, face)

4.3 Threat Model

4.3.1 Assumptions

1. Attacker has network access but not physical access to HSM
2. Attacker has access to quantum computers (post-quantum threat)
3. User devices may be compromised
4. Database may be partially leaked

4.3.2 Security Guarantees

- **Password Compromise:** Argon2id prevents rainbow table attacks
- **Session Hijacking:** HTTPOnly, Secure, SameSite cookies
- **CSRF:** Token-based protection on all state-changing operations
- **XSS:** Content Security Policy (CSP), input sanitization
- **Phishing:** WebAuthn prevents credential phishing (origin-bound credentials)
- **Quantum Attacks:** Post-quantum signatures prevent future decryption

4.3.3 Attack Surface Reduction

- Rate limiting on all authentication endpoints
- Account lockout after 5 failed login attempts
- IP-based geofencing (optional)
- User-Agent fingerprinting for anomaly detection
- Comprehensive audit logging

5 Integration with Lux Ecosystem

5.1 Lux Node

Validator and node operators authenticate via Lux ID:

- OAuth 2.0 for CLI tools (`lux-cli`)
- Client credentials flow for automated services
- DID-based node identity verification

5.2 Lux Wallet

Mobile and web wallets integrate via:

- OIDC for web wallet login
- Social recovery: Link wallet to Lux ID account for key recovery
- Multi-device synchronization: Encrypted seed phrase backup

5.3 Lux Bridge

Cross-chain identity verification:

- Verify user owns addresses on multiple chains (Bitcoin, Ethereum, Lux)
- Aggregate balances for credit scoring
- Prevent Sybil attacks in bridge operations

5.4 Lux Exchange

Trading platform authentication:

- KYC/AML compliance: Link DID to verified identity
- Trade limit enforcement based on verification level
- Withdrawal whitelist management

Algorithm 2 Cross-Chain Address Linking

- 1: User claims ownership of address on chain **X**
 - 2: Lux ID generates random challenge message
 - 3: User signs challenge with private key from chain **X**
 - 4: Lux ID verifies signature using chain **X** verification rules
 - 5: Lux ID stores (`did:lux:<address>`, `chain-X-address`) mapping
 - 6: Lux ID issues verifiable credential attesting to ownership
-

6 Cross-Chain Identity Verification

6.1 Multi-Chain Proof of Ownership

Users can link multiple blockchain addresses to a single Lux ID:

Supported Chains:

- Bitcoin (BIP-137 message signing)
- Ethereum (EIP-191 `personal_sign`)
- Polkadot (sr25519 signatures)
- Solana (Ed25519 signatures)
- Cosmos (secp256k1 via Keplr)

6.2 Decentralized Identity Aggregation

Lux ID acts as an identity aggregator:

- **ENS:** Resolve Ethereum Name Service domains
- **Unstoppable Domains:** Query `.crypto`, `.nft` domains
- **Lens Protocol:** Import social graph from Lens
- **Ceramic Network:** Integrate with Ceramic DataModels

7 Privacy Considerations

7.1 Data Minimization

Lux ID follows privacy-by-design principles:

- Collect only essential user data (email, username, password hash)
- Optional fields: phone, address, bio
- No tracking cookies or third-party analytics

7.2 Selective Disclosure

Users control which attributes to share with applications:

- OAuth scopes define requested permissions
- Users approve/deny each permission during consent
- Applications receive only approved attributes

7.3 Zero-Knowledge Proofs (Planned)

Future versions will support ZK proofs for:

- Prove age ≥ 18 without revealing birthdate
- Prove balance $\geq \$1000$ without revealing exact amount
- Prove credential issuance without revealing issuer identity

7.4 Right to be Forgotten

GDPR compliance via:

- User-initiated account deletion
- Data export in portable JSON format
- Automatic data purge after 90 days of deactivation

8 Performance and Scalability

8.1 Production Metrics (5 Years)

Metric	Value	Notes
Total Users	250,000+	Across mainnet and testnet
Daily Active Users	15,000+	6% DAU/MAU ratio
Authentication Requests	5M+/day	Peak: 12M/day
Latency (p50)	45 ms	OAuth token issuance
Latency (p99)	180 ms	Including database queries
Availability	99.95%	4.4 hours downtime/year
WebAuthn Adoption	12%	Growing 2% per quarter
Web3 Wallet Auth	8%	20,000 unique addresses

Table 2: Lux ID Production Metrics (2020-2025)

8.2 Scalability Architecture

8.2.1 Horizontal Scaling

- Stateless backend: Deploy multiple instances behind load balancer
- Session storage in Redis cluster (3-node quorum)
- Database read replicas for high-read workloads

8.2.2 Caching Strategy

- User profiles: Redis cache (5-minute TTL)
- JWKS: In-memory cache (1-hour TTL)
- DID Documents: CDN + Redis (15-minute TTL)

8.2.3 Rate Limiting

Prevent abuse via token bucket algorithm:

- Login attempts: 5 per 15 minutes per IP
- Token requests: 100 per hour per client
- DID resolution: 1,000 per hour per IP

8.3 Benchmarking

Test Setup: 8-core CPU, 16GB RAM, MySQL 8.0, Redis 7.0

Operation	Throughput	Latency (p50)
OAuth Authorization	1,200 req/s	35 ms
Token Exchange	800 req/s	52 ms
UserInfo Query	2,500 req/s	18 ms
WebAuthn Challenge	1,000 req/s	28 ms
WebAuthn Verify	600 req/s	75 ms
DID Resolution	3,000 req/s	12 ms

Table 3: Single-Instance Benchmarks

9 Comparison with Other DID Systems

Feature	Lux ID	ION	Sovrin	ENS
Web2 Compat				
OAuth/OIDC				
SAML				
WebAuthn				
Post-Quantum				
Multi-Chain				
Self-Sovereign				
Enterprise			Partial	
RBAC				
Production		Partial		
Ready				
Open Source				

Table 4: DID System Comparison

9.1 Key Differentiators

- **Lux ID:** Only system with full Web2/Web3 bridge, enterprise IAM features
- **ION (Microsoft):** Bitcoin-anchored, decentralized but lacks IAM features
- **Sovrin:** Permissioned ledger, strong privacy but complex governance
- **ENS:** Ethereum-only, excellent UX but not a full identity system

10 Future Work

10.1 Short-Term (2025-2025)

- **Passkey Support:** Implement FIDO Alliance Passkey specification
- **DID Rotation:** Allow users to rotate DIDs without losing history
- **Mobile SDK:** Native iOS/Android libraries for app integration
- **Advanced Analytics:** User behavior analysis for fraud detection

10.2 Medium-Term (2025-2027)

- **Zero-Knowledge Proofs:** Age verification, credential attestation
- **Biometric Authentication:** Face/fingerprint via WebAuthn Level 3
- **Decentralized Storage:** IPFS integration for DID Documents
- **AI-Powered Anomaly Detection:** Machine learning for security

10.3 Long-Term (2027+)

- **Quantum Key Distribution:** Integrate QKD for ultra-secure channels
- **Fully Decentralized:** Transition to permissionless DID registry
- **Interplanetary Identity:** Identity system for Mars colonization
- **Neural Interface:** Thought-based authentication (research phase)

11 Conclusion

Lux ID demonstrates that decentralized identity can coexist with enterprise IAM requirements. Over five years, the system has evolved from a Casdoor fork to a production-grade, multi-protocol identity platform supporting 250,000+ users and 5M+ daily authentication requests. The October 2025 revision introduces post-quantum cryptography, ensuring security against future quantum computing threats.

The `did:lux:address` format provides a blockchain-native identity standard while maintaining compatibility with OAuth, SAML, LDAP, and other Web2 protocols. This pragmatic hybrid approach enables gradual migration to self-sovereign identity without sacrificing user experience or enterprise features.

As blockchain ecosystems mature, identity systems like Lux ID will become critical infrastructure for bridging traditional and decentralized web paradigms. Our architecture, security model, and integration patterns provide a blueprint for the next generation of universal identity platforms.

Acknowledgments

We thank the Casdoor project for providing the foundational IAM codebase. Special thanks to the Lux Foundation engineering team, identity working group, and 250,000+ users who have provided feedback over five years. This work was supported by the Lux Foundation.

References

- [1] W3C Decentralized Identifiers (DIDs) v1.0. *W3C Recommendation*, 19 July 2022. <https://www.w3.org/TR/did-core/>
- [2] Decentralized Identity Foundation. *Universal Resolver*. <https://dev.uniresolver.io/>
- [3] D. Hardt. *The OAuth 2.0 Authorization Framework*. RFC 6749, October 2012. <https://tools.ietf.org/html/rfc6749>
- [4] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, C. Mortimore. *OpenID Connect Core 1.0*. OpenID Foundation, November 2014. https://openid.net/specs/openid-connect-core-1_0.html
- [5] W3C Web Authentication Working Group. *Web Authentication: An API for accessing Public Key Credentials Level 3*. W3C Editor's Draft, 2025. <https://www.w3.org/TR/webauthn-3/>
- [6] L. Ducas et al. *CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme*. NIST PQC Standardization, 2024. <https://pq-crystals.org/dilithium/>
- [7] R. Avanzi et al. *CRYSTALS-Kyber: A CCA-Secure Module-Lattice-Based KEM*. NIST PQC Standardization, 2024. <https://pq-crystals.org/kyber/>
- [8] D. M'Raihi, S. Machani, M. Pei, J. Rydell. *TOTP: Time-Based One-Time Password Algorithm*. RFC 6238, May 2011. <https://tools.ietf.org/html/rfc6238>
- [9] A. Biryukov, D. Dinu, D. Khovratovich. *Argon2: The Memory-Hard Function for Password Hashing*. Password Hashing Competition Winner, 2015. <https://github.com/P-H-C/phc-winner-argon2>
- [10] Casbin Organization. *Casdoor: A UI-first Identity and Access Management (IAM) / Single-Sign-On (SSO) platform*. <https://casdoor.org/>
- [11] Microsoft Identity Division. *ION: A Public, Permissionless, Decentralized Identifier Network*. <https://identity.foundation/ion/>
- [12] Sovrin Foundation. *Sovrin: A Protocol and Token for Self-Sovereign Identity*. White Paper, 2018. <https://sovrin.org/>
- [13] Nick Johnson et al. *Ethereum Name Service: The Distributed, Open, and Extensible Naming System*. ENS Documentation, 2023. <https://docs.ens.domains/>

A DID Method Specification

A.1 Method Name

The method name is `lux`.

A.2 Method-Specific Identifier

The method-specific identifier is a Lux blockchain address in one of the following formats:

- Bech32 (X-Chain/P-Chain): `[XP]-lux1[a-z0-9]{38}`
- Hex (C-Chain): `0x[0-9a-fA-F]{40}`

A.3 DID Syntax

```
did-lux = "did:lux:" network ":" address
network = "mainnet" / "testnet" / "local" / appchain-id
address = bech32-address / hex-address
bech32-address = ([XP] "-lux1") 1*38(ALPHA / DIGIT)
hex-address = "0x" 40(HEXDIG)
appchain-id = 32(base58-char)
```

A.4 CRUD Operations

A.4.1 Create

Request:

```
1 POST /api/did/create
2 Content-Type: application/json
3
4 {
5   "address": "X-lux1qzr2v3dhq0hgqkgdwq36z0z7eqkh5x2g5m",
6   "publicKey": "zH3C2AVvLMv6gmMnam3uVAjZpfkcJCwDwnZn6z3wXmqPV",
7   "verificationMethod": "Ed25519VerificationKey2020"
8 }
```

Response:

```
1 {
2   "did": "did:lux:X-lux1qzr2v3dhq0hgqkgdwq36z0z7eqkh5x2g5m",
3   "didDocument": { /* Full DID Document */ },
4   "txHash": "0xabc...def" // If on-chain registration
5 }
```

A.4.2 Read (Resolve)

Request:

```
1 GET /api/did/resolve/did:lux:X-lux1qzr2v3dhq0hgqkgdwq36z0z7eqkh5x2g5m
```

Response:

```
1 {
2   "didDocument": { /* Full DID Document */ },
3   "didResolutionMetadata": {
4     "contentType": "application/did+ld+json",
5     "retrieved": "2025-10-28T12:00:00Z"
6   },
7   "didDocumentMetadata": {
8     "created": "2020-10-20T10:30:00Z",
9     "updated": "2025-10-15T14:22:00Z",
10    "versionId": "5"
11  }
12 }
```

A.4.3 Update

Request:

```
1 POST /api/did/update
2 Content-Type: application/json
3 Authorization: Bearer <jwt-signed-by-did-key>
4
5 {
6   "did": "did:lux:X-lux1qzr2v3dhq0hgqkgdwq36z0z7eqkh5x2g5m",
```

```

7  "updates": [
8    {
9      "action": "addVerificationMethod",
10     "verificationMethod": {
11       "id": "#key-4",
12       "type": "Dilithium3VerificationKey2025",
13       "publicKeyMultibase": "zMxY...PQ-KEY"
14     }
15   }
16 ]
17 }

```

A.4.4 Deactivate

Request:

```

1 POST /api/did/deactivate
2 Content-Type: application/json
3 Authorization: Bearer <jwt-signed-by-did-key>
4
5 {
6   "did": "did:lux:X-lux1qzr2v3dhq0hgqkgdwq36z0z7eqkh5x2g5m",
7   "reason": "User requested account deletion"
8 }

```

B API Reference

B.1 Authentication Endpoints

```

1 # OAuth 2.0
2 GET /api/authorize
3 POST /api/token
4 POST /api/revoke
5 GET /api/userinfo
6
7 # OpenID Connect
8 GET /.well-known/openid-configuration
9 GET /.well-known/jwks.json
10
11 # SAML
12 POST /api/saml/acs # Assertion Consumer Service
13 GET /api/saml/metadata # IdP Metadata
14 POST /api/saml/slo # Single Logout
15
16 # WebAuthn
17 GET /api/webauthn/signup/begin # Registration start
18 POST /api/webauthn/signup/finish # Registration complete
19 GET /api/webauthn/signin/begin # Authentication start
20 POST /api/webauthn/signin/finish # Authentication complete
21
22 # Web3 Wallet
23 POST /api/web3/challenge # Get challenge nonce
24 POST /api/web3/verify # Verify signature

```

B.2 Management Endpoints

```

1 # User Management
2 GET /api/get-users
3 POST /api/add-user

```

```

4 POST    /api/update-user
5 POST    /api/delete-user
6
7 # Application Management
8 GET      /api/get-applications
9 POST     /api/add-application
10 POST    /api/update-application
11 POST    /api/delete-application
12
13 # Organization Management
14 GET      /api/get-organizations
15 POST     /api/add-organization
16 POST     /api/update-organization
17 POST     /api/delete-organization
18
19 # DID Operations
20 POST     /api/did/create
21 GET      /api/did/resolve/:did
22 POST     /api/did/update
23 POST     /api/did/deactivate

```

C Deployment Guide

C.1 Docker Compose (Development)

```

1 # compose.yaml
2 services:
3   lux-id:
4     image: ghcr.io/luxfi/id:latest
5     ports:
6       - "8000:8000"
7     environment:
8       - MYSQL_HOST=db
9       - MYSQL_PORT=3306
10      - MYSQL_DATABASE=lux_id
11      - REDIS_HOST=redis
12      - REDIS_PORT=6379
13     depends_on:
14       - db
15       - redis
16
17   db:
18     image: mysql:8.0
19     environment:
20       - MYSQL_ROOT_PASSWORD=secret
21       - MYSQL_DATABASE=lux_id
22     volumes:
23       - mysql_data:/var/lib/mysql
24
25   redis:
26     image: redis:7.0
27     volumes:
28       - redis_data:/data
29
30 volumes:
31   mysql_data:
32   redis_data:

```

C.2 Kubernetes (Production)

```
1 # k8s/deployment.yaml
2 apiVersion: apps/v1
3 kind: Deployment
4 metadata:
5   name: lux-id
6 spec:
7   replicas: 3
8   selector:
9     matchLabels:
10      app: lux-id
11   template:
12     metadata:
13       labels:
14         app: lux-id
15     spec:
16       containers:
17       - name: lux-id
18         image: ghcr.io/luxfi/id:v2025.10
19         ports:
20         - containerPort: 8000
21         env:
22         - name: MYSQL_HOST
23           value: "mysql-service"
24         - name: REDIS_HOST
25           value: "redis-service"
26       resources:
27         requests:
28           memory: "512Mi"
29           cpu: "500m"
30         limits:
31           memory: "1Gi"
32           cpu: "1000m"
```

Lux ID Frontend
(React 19 + TypeScript + Radix UI + Black Theme)

API Gateway (Beego)

- Request routing
- Authentication middleware
- Rate limiting & CSRF protection

OAuth/OIDC	SAML/CAS	WebAuthn/MFA
Controllers	Controllers	Controllers

Business Logic Layer

- User Management
- Token Generation/Validation
- Permission Enforcement (Casbin RBAC)
- Verification (Email, Phone, WebAuthn)
- Wallet Authentication (MetaMask, WalletConnect)

MySQL/ PostgreSQL	Redis (Sessions)	Blockchain (Lux C-Chain)
----------------------	---------------------	-----------------------------

Figure 1: Lux ID System Architecture